Data Platform Engineer - AWS DevOps - rozmowa 60 minut

Materiał dla prowadzącego. Klucz odpowiedzi zachowaj oddzielnie od arkusza kandydata. Podczas zadania skorzystaj z osobnego arkusza. Przykładowe odpowiedzi i kalibracja poziomu pomogą rozdzielić wymagania regular/mid od mocniejszych odpowiedzi dodatkowych.

Podstawa: ogłoszenie i trzy dostarczone zrzuty CV. Polska wersja: 17 października 2026. To tłumaczenie do przygotowania prowadzącego. **Cała właściwa rozmowa nadal odbywa się po angielsku**: otwarcie, pytania techniczne, zadanie, dopytania i zakończenie. Komunikację oceniaj przez cały czas; nie ma osobnego testu językowego ani zmiany języka.

1. Cel i dopasowanie do roli

Profil rozmowy to **Data Platform Engineer - AWS DevOps**, zgodnie z doprecyzowaniem prowadzącego. Sprawdź, czy kandydat potrafi budować, wdrażać, zabezpieczać i utrzymywać platformę danych AWS używaną przez data engineerów i odbiorców danych. Python służy automatyzacji; wynikiem pracy jest niezawodna, obserwowalna platforma wdrażana powtarzalnie.

Odpowiedzialność	Jakich dowodów szukać
Platforma danych AWS	Rozumie dostarczanie danych, S3, przetwarzanie, katalogowanie i zapytania
IaC i standardy platformy	Tworzy moduły wielokrotnego użytku, rozdziela środowiska, zarządza stanem i zmianami
CI/CD	Wdraża infrastrukturę i artefakty zadań z testami oraz kontrolą promocji
Bezpieczeństwo i sieć	Rozumie IAM, KMS, dostęp między kontami i łączność z prywatnych podsieci
Utrzymanie danych	Mierzy świeżość i kompletność; obsługuje incydenty, ponowienia i uzupełnianie historii
Wsparcie data engineerów	Zapewnia wzorce wdrożeń, monitoring i instrukcje operacyjne

Nie wymagaj implementacji złożonych transformacji Spark w tej godzinie. Pytaj, jak zadania są wdrażane, uruchamiane, zabezpieczane i monitorowane. Akceptuj inną architekturę, jeśli kandydat potrafi uzasadnić, jak spełnia ona wymagania.

Ogłoszenie wymaga odpowiedniego doświadczenia, pracy w chmurze, programowania, IaC i komunikacji. Kontenery i konkretne platformy danych to atuty; rozwijaj je, jeżeli mają znaczenie w rzeczywistym projekcie. Ogłoszenie Capgemini.

Co zweryfikować na podstawie CV

Deklaracja z CV	Co sprawdzić
Migracja S3 między kontami z odpowiedzialnością end-to-end	Osobisty wkład, kontrola kompletności, przełączenie odbiorców, obsługa błędów
Python, boto3, pytest, logowanie strukturalne	Krótki przegląd kodu ustnie i konkretny przykład automatyzacji oraz sprawdzania jej działania
Rozbudowa Terraform i CloudFormation	Stan, plany, drift i częściowo nieudane wdrożenia
GitHub Actions, OIDC, testy po wdrożeniu	Uwierzytelnianie, granice uprawnień i przydatne testy integracyjne
Rotacja kluczy IAM przez Lambda i Secrets Manager	Częściowe awarie, ponowienia, odbiorcy sekretów i odzyskiwanie procesu
Platforma objęta GxP i change management	Konkretne dowody przeglądów, testów, akceptacji i audytu
Operacje finansowe i kierowanie zespołem	Jasna komunikacja, diagnozowanie problemów i zakres wcześniejszej pracy technicznej

CV wskazuje rolę Cloud Engineer od lipca 2025. Ogłoszenie wymaga co najmniej trzech lat w podobnej roli. Doprecyzuj wcześniejsze obowiązki techniczne, a formalny wymóg stażu potwierdź osobno z rekruterem. Daty i tytuły stanowisk same nie przesądzają o kompetencjach. Certyfikaty i lata w operacjach finansowych nie zastępują dowodów pracy inżynierskiej.

Nie zakładaj braku umiejętności tylko dlatego, że nie widać ich na trzech zrzutach. Oddzielnie zapisuj potwierdzone przykłady i obszary niesprawdzone.

2. Agenda - dokładnie 60 minut

Minuty	Blok	Główny cel
0-3	Wprowadzenie	Wyjaśnić rolę AWS DevOps dla platformy danych i format rozmowy
3-7	Q1: własny projekt	Ustalić, co kandydat osobiście dostarczył i utrzymywał
7-17	Q2: platforma AWS i bezpieczeństwo	Omówić przepływ, dostęp, przetwarzanie i ponowienia
17-27	Q3: Terraform	Omówić stan, środowiska i częściowy błąd wdrożenia
27-34	Q4: CI/CD i OIDC	Bezpiecznie wdrażać infrastrukturę i artefakty przetwarzania
34-44	Q5: Python	Ustnie przejrzeć dwie krótkie funkcje i omówić rzeczywistą automatyzację
44-55	Q6: operacje i sieć	Zdiagnozować incydent, odtworzyć dane i przekazać informację o wpływie
55-60	Pytania kandydata	Omówić projekt, odpowiedzialność i kolejne kroki

Zadaj pytanie główne, a dopytania dobieraj do odpowiedzi i pozostałego czasu. Listy dopytań są bankiem pytań, a nie obowiązkową listą do odczytania. Zachowaj ostatnie pięć minut na pytania kandydata.

Proponowane otwarcie - tłumaczenie

> Dziękuję za udział. Mamy 60 minut i całą rozmowę przeprowadzimy po angielsku. Porozmawiamy o Twoich projektach AWS, projektowaniu platformy, Terraformie, pipeline'ach wdrożeniowych i diagnozowaniu problemów produkcyjnych. Będzie również krótki, ustny przegląd kodu Python, bez obowiązku pisania kodu na żywo. Interesuje mnie, co robiłeś osobiście, dlaczego podejmowałeś konkretne decyzje i jak sprawdzałeś wynik. Przybliżone, zanonimizowane przykłady są w porządku. Przy zadaniu możesz korzystać z dokumentacji API. Ostatnie pięć minut zostawimy na Twoje pytania.

Przydatne dopytania przez całą rozmowę:

- „Czy możesz przeprowadzić mnie przez konkretny przykład?”
- „Którą część zaimplementowałeś samodzielnie?”
- „Jaki dowód potwierdziłby tę hipotezę?”
- „Co się stanie, jeśli operacja przerwie się w połowie?”
- „Jak wyjaśniłbyś to data engineerowi albo odbiorcy biznesowemu?”

3. Pytania i wskazówki dla prowadzącego

Q1 - odpowiedzialność za projekt AWS, 4 minuty

Zapytaj:

> W CV piszesz o migracji S3 między kontami z odpowiedzialnością end-to-end. Czy możesz naszkicować przepływ danych, wskazać odbiorców oraz elementy, które osobiście zaprojektowałeś, zaimplementowałeś lub utrzymywałeś po wdrożeniu?

Wybierz dopytanie:

- Ile w przybliżeniu było kont, obiektów i danych?
- Co już istniało, gdy dołączyłeś, a co dostarczyli inni członkowie zespołu?
- Opisz jedną własną decyzję i odrzuconą alternatywę.
- Jaki problem był najtrudniejszy i jak potwierdziłeś jego przyczynę?
- Jaką automatyzację techniczną tworzyłeś przed obecną rolą i kto jej używał?

Mocne dowody: rozdziela wkład własny i zespołowy; wyjaśnia przepływ i odbiorców; podaje konkretną zmianę, test i wynik; przyznaje, co było poza jego zakresem.

Jeśli nadal wymienia jedynie usługi lub działania zespołu, poproś o jeden konkretny przykład. Samo używanie słowa „my” nie jest sygnałem negatywnym.

Q2 - platforma danych AWS i bezpieczeństwo, 10 minut

Scenariusz główny - około 6 minut:

> Dostawca wysyła pliki CSV i JSON do S3. Data engineerowie mają je przetwarzać, a analitycy odpytywać przygotowane dane. Mamy oddzielne środowiska dev, test i prod. Nowe dane mają być gotowe do odczytu w ciągu 15 minut od dostarczenia. Jak przygotujesz i będziesz utrzymywać tę platformę AWS? Najpierw zapytaj o brakujące wymagania, a potem naszkicuj przepływ.

To scenariusz rekrutacyjny, a nie potwierdzona architektura Capgemini. Cel 15 minut jest założeniem ćwiczenia. Nie oceniaj doboru zasobów bez podania wolumenów danych i wymagań dotyczących transformacji.

Wybierz dwa dopytania:

- Gdzie zachowasz niezmienione dane źródłowe, a gdzie przygotowane zbiory?
- Co uruchamia przetwarzanie? Jak obsłużysz buforowanie i zależności między etapami?

- Jak wybrałbyś Lambdę lub Glue do tego obciążenia?
- Co, jeśli dostawca ponownie wyśle ten sam plik albo zmieni jego schemat?
- Jak uzupełnisz dane za jeden dzień bez ich podwójnego doliczenia?
- Jaki gotowy moduł lub wzorzec wdrożenia dasz data engineerowi, żeby nowe zadanie nie wymagało za każdym razem ręcznego konfigurowania infrastruktury?

Mocne dowody: pyta o wolumen, częstotliwość, odbiorców, bezpieczeństwo i koszt; rozdziela źródła od wyników; uzasadnia wybór przetwarzania i wyzwalania; uwzględnia schemat, walidację, izolowanie błędnych danych, historię uruchomień i bezpieczne ponowienia. Potrafi wskazać zasoby definiowane w IaC i elementy wymagające monitorowania.

Przykładowy kierunek: S3 raw → wyzwalanie i opcjonalny bufor zdarzeń → przetwarzanie → S3 curated → katalog i zapytania. Glue może zapewniać przetwarzanie i Data Catalog; Athena jest przykładem warstwy zapytań. Nie wymagaj użycia każdej usługi ani tej jednej kombinacji. AWS: Glue i powiązane usługi.

Bezpieczeństwo - około 3 minut:

> Zadanie działa w koncie platformy, a analityk używa innego konta AWS. Dane są szyfrowane kluczem KMS. Jak ograniczysz analityka do zatwierdzonego zbioru i jak zdiagnozujesz AccessDenied podczas odczytu?

Szukaj rozdzielenia ról wykonawczych i odbiorców, polityk IAM, polityk zasobów S3 oraz uprawnień i polityk KMS. Kandydat powinien ustalić faktyczną tożsamość, operację, konto i zasób. Jeśli zaproponuje Lake Formation, dopytaj o jej dodatkową warstwę uprawnień; nie traktuj jej jako niejawnego warunku koniecznego. AWS: diagnostyka dostępu i szyfrowania.

Ostatnia minuta:

> Co mogłoby generować największe koszty w Twoim projekcie i jak sprawdzisz, że optymalizacja nie pogarsza czasu dostarczania danych?

Szukaj kosztów powiązanych z obciążeniem, pomiarów i kompromisów względem celu. Nie pytaj o ceny usług z pamięci. Same nazwy usług nie dowodzą zrozumienia platformy.

Q3 - Terraform w zespole, 10 minut

Zapytaj:

> Rozbudowujesz moduł Terraform o buckety raw i curated, role IAM, zadanie przetwarzania i alarmy dla dev, test i prod. Przeprowadź mnie przez zmianę od pull requesta do produkcji. Gdzie przechowujecie stan i jak zapobiegacie konfliktom zmian dwóch osób?

Dopytanie obowiązkowe:

> Apply utworzył bucket, ale nie udało się skonfigurować jego polityki. Co sprawdzisz przed następnym uruchomieniem? Czy cofnięcie commita cofa infrastrukturę?

Jeśli jest czas, wybierz jedno dopytanie:

- Jak rozdzielisz stan i konfigurację środowisk oraz ograniczysz skutki pomyłki?
- Ktoś zmienia zasób w konsoli. Jak wykryjesz drift i zdecydujesz, którą konfigurację zachować?
- Plan przewiduje usunięcie zasobu z danymi. Co sprawdzisz przed kontynuowaniem?
- Co może być zapisane w state? Czy `sensitive = true` usuwa sekret ze stanu?

Oczekiwane rozumowanie: rozumie rolę stanu i granice planu; omawia kontrolowany zdalny backend, ograniczenia dostępu, ochronę stanu, zmiany destrukcyjne i rozdzielenie środowisk. Po częściowej awarii sprawdza rzeczywiste zasoby i state, bada przyczynę, naprawia ją i przegląda nowy plan. Terraform nie zapewnia transakcyjnego wycofania całego apply. Sam git revert nie zmienia AWS. Sekrety mogą pozostać w stanie, mimo oznaczenia ich jako sensitive. HashiCorp: dane wrażliwe.

Oddziel blokowanie stanu od serializacji całego procesu wdrożeniowego. Backend S3 obsługuje opcjonalną blokadę przez `use_lockfile`; starsza blokada oparta na DynamoDB jest oznaczona jako deprecated. Rzetelny opis starszej konfiguracji z DynamoDB pozostaje wiarygodnym dowodem doświadczenia. HashiCorp: backend S3.

Dopytaj przy propozycjach usuwania state, wymuszania odblokowania bez sprawdzenia właściciela blokady albo założeniu, że kolejne apply odtworzy wszystko od początku. Zanim wyciągniesz wniosek, poproś o konkretne kroki.

Q4 - CI/CD, OIDC i testy wdrożeniowe, 7 minut

Zapytaj:

> W CV wymieniasz OIDC i sprawdzenia pytest po wdrożeniu. Opisz pipeline od PR do produkcji. Jak wdrażasz Terraform i wersjonowany skrypt Lambda lub zadanie Glue, co testujesz, kto zatwierdza zmianę i jak zadanie CI uzyskuje dostęp do AWS?

Wybierz dopytania:

- Co ogranicza przyjęcie roli do właściwego repozytorium i kontekstu wdrożenia?
- Czym trust policy różni się od uprawnień nadanych roli?
- Jaki test wykryje, że zasoby istnieją, ale właściwy odbiorca nadal nie może czytać danych?
- Jak przed promocją sprawdzisz nową wersję zadania na reprezentatywnych danych i schemacie?
- Co zrobisz, jeśli apply zakończyło się sukcesem, a test integracyjny nie?

Oczekiwane rozumowanie: krótkotrwałe poświadczenia przez OIDC/STS, uprawnienie do pobrania tokenu w zadaniu i ograniczona relacja zaufania. Znaczenie mają m.in. `aud` i `sub` oraz zabezpieczenia środowiska produkcyjnego. Nie wymagaj dokładnego formatu `sub` z pamięci: zależy od konfiguracji repozytorium i tokenu. Samo `id-token: write` nie daje dostępu do zasobów AWS. GitHub: OIDC w AWS.

Szukaj odróżnienia mocków od integracji: jaka tożsamość uruchamia test, czy odpowiada odbiorcy, jakie zasoby testowe tworzy, jak sprząta i czy sprawdza odmowę dostępu. Wycofanie infrastruktury nie zawsze jest bezpieczne dla danych. Akceptuj uzasadnioną naprawę w przód, wycofanie aplikacji albo zatrzymanie promocji zależnie od skutków zmiany.

Sprawdź zgodność wersji infrastruktury, konfiguracji i artefaktów zadań. Kandydat powinien wyjaśnić, co jest promowane między środowiskami i jak wadliwa wersja zostaje zatrzymana przed produkcją.

Q5 - krótki przegląd Pythona i doświadczenie z automatyzacją, 10 minut

Otwórz arkusz kandydata.

Format: wyłącznie ustny. Edytor, konfiguracja SDK AWS, pełna implementacja, mocki i pisanie testów nie są wymagane. Są dwa krótkie zadania: cztery minuty na return i wspólną wartość domyślną, dwie na warunek logiczny i cztery na własne doświadczenie z automatyzacją. Wpisywanie kodu jest opcjonalne. Rekordy zadań to dane przykładowe, a nie surowa odpowiedź API AWS. W zadaniu 1 każde wywołanie ma dawać niezależny wynik.

Zadanie 1 - pierwsze 4 minuty:

> Funkcja powinna zwracać nazwy wszystkich nieudanych zadań. Co rzeczywiście zwraca dla przykładowych danych? Jaka zmiana pozwoli sprawdzić wszystkie zadania? Po wprowadzeniu tylko tej zmiany, co zwrócą dwa kolejne wywołania failed_jobs(jobs)? Przyjmij świeżą definicję funkcji i brak innych wywołań. Wyjaśnij drugi błąd i poprawkę. Po naprawieniu obu błędów, co powinna zwrócić dla pustej listy albo gdy nie ma nieudanych zadań?

Pierwszy rekord ma status sukcesu, a dwa następne status błędu. Obecna funkcja znajduje się w arkuszu.

Oczekiwane rozumowanie - oba błędy są wymagane:

- `return` jest wewnątrz pętli, więc funkcja kończy się po pierwszym rekordzie. Przy świeżej liście domyślnej oryginalny przykład zwraca `[]`, a puste wejście `None`.
- Samo przeniesienie `return failed` poza pętlę pozostawia współdzielone `failed=[]` przy pominięciu drugiego argumentu. Pierwsze wywołanie dla przykładu zwraca `["load_orders", "load_products"]`, a drugie od razu

`["load_orders", "load_products", "load_orders", "load_products"]`. Lista domyślna powstaje przy definiowaniu funkcji, a `append` ją modyfikuje. Nawet późniejsze puste wejście lub same sukcesy mogą zwrócić stare błędy.

- Usuń parametr akumulatora i utwórz `failed = []` wewnątrz każdego wywołania. Po obu poprawkach puste wejście i same sukcesy zwracają `[]` niezależnie od historii. Dopuszczalny jest też domyślny `None` i nowa lista w każdym wywołaniu.

Pierwotny przedwczesny return ukrywa akumulację w tych danych, bo pierwszy rekord jest udany. Pytanie o kolejne wywołania zadaj po omówieniu poprawki wcięcia. Przed drugą poprawką oba wywołania zwracają ten sam obiekt listy; zapisany pierwszy wynik zmienia się po dopisaniu w drugim wywołaniu. To dodatkowa głębia odpowiedzi.

Kod referencyjny tylko dla prowadzącego:

```
def failed_jobs(jobs):
    failed = []
    for job in jobs:
        if job["status"] == "FAILED":
            failed.append(job["name"])
    return failed
```

Zadanie 2 - kolejne 2 minuty: zadania do ponowienia

Otwórz zadanie 2 w arkuszu. Funkcja wybiera uruchomienia `FAILED` lub `TIMED_OUT`, ale używa `if job["status"] == "FAILED" or "TIMED_OUT":`.

> Co zwróci retryable_jobs(runs), a co powinna zwrócić? Dlaczego warunek wybiera również zadania udane lub anulowane? Wyjaśnij drobną poprawkę i przykładowe wejścia do jej sprawdzenia. Nie trzeba nic implementować.

Oczekiwane rozumowanie: niepusty napis `"TIMED_OUT"` jest traktowany jako prawda; `or` nie powtarza porównania automatycznie. Każdy rekord spełnia warunek. Wynik to `["load_customers", "load_orders", "load_products", "load_prices"]`, a oczekiwany wynik to `["load_orders", "load_products"]`. Akceptuj sprawdzenie przynależności lub jawne porównanie po obu stronach `or`.

```
def retryable_jobs(jobs):
    retryable = []
    for job in jobs:
        if job["status"] in ("FAILED", "TIMED_OUT"):
            retryable.append(job["name"])
    return retryable
```

Sprawdź mieszane statusy, każdy status dozwolony i pomijany oraz puste wejście. Nie wymagaj mechanizmu ponowień, znajomości SDK ani pisania testów. Przy trudnościach zapytaj, co oznacza niepusty napis w `if`, i zapisz zakres podpowiedzi.

Pozostałe 4 minuty - rzeczywista automatyzacja:

> Opisz jedną automatyzację w Pythonie, którą osobiście zaimplementowałeś. Co robiła, jak obsługiwałeś błędy i jak sprawdzałeś ją przed wdrożeniem?

Wybierz jedno dopytanie:

- Jak nieudane albo częściowo udane uruchomienie było widoczne dla wywołującego lub operatora?
- Jakie przypadki sprawdzałeś i jak izolowałeś usługi zewnętrzne w testach?
- Jeśli wywołanie API przekroczy limit czasu, jak ocenisz bezpieczeństwo ponowienia?
- Jaki rzeczywisty błąd znalazłeś i jakie dowody potwierdziły poprawkę?

Wystarczające na regular/mid: wyjaśnia przedwczesny return i akumulację między wywołaniami w zadaniu 1, proponuje niezależne wyniki i wyjaśnia wybór wszystkich rekordów oraz poprawkę warunku w zadaniu 2. Podaje konkretny przykład automatyzacji i sprawdzania błędów. Oba zadania są częścią rozmowy; pytanie o kolejne wywołania pozostaje obowiązkowe. Zapisz podpowiedzi potrzebne do wyjaśnienia wspólnej wartości domyślnej lub warunku logicznego. List comprehension jest dopuszczalne, ale niewymagane. Jeśli szybko wyjaśni wszystkie błędy, przejdź dalej; nie wypełniaj czasu ciekawostkami składniowymi.

Dodatkowa głębokość: na własnym przykładzie wyjaśnia bezpieczne ponowienia, strukturalne raportowanie błędów lub różnicę między testem jednostkowym a integracyjnym. Nie wymagaj implementowania tych mechanizmów podczas rozmowy.

Oba fragmenty są krótką kontrolą czytania kodu. Samo znalezienie tych błędów nie potwierdza mocnego doświadczenia programistycznego. Głębokość oceniaj przez dowody projektowe. Oddziel wykazane czytanie kodu od deklarowanego doświadczenia implementacyjnego.

Q6 - operacje, sieć i komunikacja incydentu, 11 minut

Diagnoza incydentu - około 4 minut:

> Wróć do projektu z Q2 albo przyjmij S3 → SQS → Lambda → zbiór do raportowania. Po wdrożeniu CI jest zielone i pliki trafiają do S3, ale raport od 40 minut nie ma nowych danych. Jakie są Twoje pierwsze trzy sprawdzenia? Co potwierdzi albo odrzuci Twoją główną hipotezę?

Szukaj określenia wpływu, ostatniego poprawnego przetworzenia, porównania z ostatnią zmianą i prześledzenia konkretnego obiektu lub wiadomości. Dowodami mogą być: dostarczenie zdarzenia, liczba i wiek wiadomości w kolejce, DLQ, wywołania i błędy Lambdy, logi, zapis wyniku i świeżość danych odbiorcy. Brak błędów Lambdy może oznaczać, że nie jest w ogóle wywoływana.

Odtworzenie przetwarzania i monitoring - około 3 minut:

> Ta sama wiadomość trafia do Lambdy dwa razy. Jak unikniesz podwójnego skutku biznesowego? Co, jeśli jedna wiadomość z paczki jest błędna? Jak po naprawie odtworzysz zaległe dane i potwierdzisz kompletność?

SQS/Lambda wymaga uwzględnienia ponowień. Szukaj stabilnego klucza operacji i trwałego śledzenia skutku lub idempotentnego zapisu; zbiór w pamięci procesu nie wystarcza. Częściowe odpowiedzi dla paczek, DLQ i kontrolowany replay to przydatne opcje; nie wymagaj formatu JSON odpowiedzi z pamięci. AWS: Lambda z SQS.

Zadaj jedno krótkie pytanie o monitoring:

> Jaki alarm wykryje nieaktualne lub niekompletne dane mimo sukcesu wszystkich zadań i kto będzie odpowiedzialny za reakcję?

Szukaj miernika świeżości/kompletności powiązanego z celem, właściciela i runbooka. Naprawa musi obejmować wynik danych, a nie sam restart usługi. Wycofanie wdrożenia nie musi odzyskać utraconych danych ani usunąć skutków podwójnego przetwarzania. AWS: doskonałość operacyjna w analityce.

Sieć AWS - około 2 minut:

> Po przeniesieniu Lambdy do prywatnych podsieci wywołania API kończą się timeoutem. Jak odróżnisz problemy sieciowe od IAM i sprawdzisz dostęp do S3 oraz Secrets Manager?

Szukaj ustalenia usługi docelowej, tras, DNS, reguł sieci i konfiguracji endpointu lub NAT zależnie od potrzeb. Umieszczenie Lambdy w publicznej podsieci nie daje automatycznie dostępu do internetu. Odróżniaj brak łączności od AccessDenied; polityka endpointu również może ograniczać dostęp. Nie wymagaj NAT, jeśli wystarczy odpowiedni endpoint. AWS: łączność Lambdy w VPC.

Informacja dla odbiorcy biznesowego - ostatnie 2 minuty:

> Przekaż krótki status incydentu odbiorcy biznesowemu. Wyjaśnij wpływ, co już wiadomo, co pozostaje niepewne, jakie są kolejne działania i kiedy przekażesz następną aktualizację.

Jeśli jest czas, zastąp jedno dopytanie następującym:

> Odbiorca biznesowy prosi o pominięcie zwykłego procesu wdrożenia, żeby szybciej przywrócić usługę. Jak odpowiesz w środowisku regulowanym?

Oceń jasność, strukturę, oddzielenie faktów od hipotez i realistyczną deklarację kolejnej aktualizacji. To zadanie komunikacyjne w ramach części technicznej, a nie osobny test języka. Przy GxP szukaj opisu rzeczywistej ścieżki awaryjnej zmiany, testów, zgód i dowodów, a nie przepisów wyuczonych na pamięć.

Minuty 55-60 - pytania kandydata i zakończenie

Oddaj kandydatowi pozostały czas. Przygotuj informacje o faktycznym stosie AWS, odpowiedzialności zespołu, dyżurach, review, samodzielności i wdrożeniu do pracy. Zapisz pytania, na które nie znasz odpowiedzi, do późniejszego wyjaśnienia. Brak pytań interpretuj w kontekście tego, co już zostało powiedziane.

Proponowane zakończenie - tłumaczenie:

> Omówiliśmy główne zagadnienia techniczne. Czego chciałbyś dowiedzieć się o platformie, zespole albo tej roli?
>
> Dziękuję za omówienie doświadczeń i zadania. Kolejny krok to [wstaw uzgodniony proces rekrutacyjny]. Skontaktujemy się przez [wstaw uzgodniony kanał kontaktu].

W miejsce nawiasów wpisuj wyłącznie potwierdzone ustalenia rekrutacyjne.

4. Arkusz oceny - wypełnij po rozmowie

Oceniaj względem samodzielności potrzebnej w tym zespole. To propozycja rubryki do uzgodnienia z zespołem, a nie oficjalny próg zatrudnienia Capgemini.

Ocena	Obserwowalne dowody
1	Nie rozwiązuje ani nie wyjaśnia podstawowego problemu mimo doprecyzowania
2	Rozwiązuje podstawy z podpowiedziami; wymaga wsparcia przy wdrożeniu i błędach
3	Samodzielnie rozwiązuje typowy problem, sprawdza wynik i rozpoznaje ryzyka
4	Wyjaśnia kompromisy, częściowe awarie i weryfikację, popierając je konkretnym doświadczeniem
N/O	Nie oceniono: brak czasu lub dowodów; nie zastępuj zerem

Obszar	Proponowana waga	Ocena 1-4 / N/O	Dowody i potrzebne wsparcie
Platforma danych AWS, IAM/KMS i sieć	20%		
Terraform / IaC i zarządzanie środowiskami	20%		
Operacje na danych, diagnoza i odtwarzanie	20%		
CI/CD infrastruktury i zadań przetwarzania	15%		
Czytanie Pythona, automatyzacja i podejście do testów	15%		
Własny wkład i komunikacja po angielsku	10%		

Wagi porządkują dowody; nie podejmuj decyzji tylko na podstawie średniej. Jeśli rola wymaga samodzielności, szukaj poziomu 3 w AWS/IaC i operacjach na danych. Pythona oceniaj przez przegląd kodu i konkretne przykłady projektowe. Przy poziomie 2 zapisz potrzebne wsparcie i oceń, czy zespół może je zapewnić. Jeśli kluczowy obszar ma N/O, zbierz więcej dowodów przed rekomendacją.

Ocena komunikacji przez całą rozmowę

Obserwuj, czy kandydat umie po angielsku doprecyzować wymagania, wyjaśnić diagram i kod, omówić kompromisy, zrozumieć dopytania i zwięźle opisać incydent. Oceniaj jasność i rozumienie potrzebne w pracy, nie akcent ani pojedyncze błędy gramatyczne. Wyjaśnij niejednoznaczne sformułowanie przed uznaniem go za brak techniczny; oddziel trudność językową od luki w wiedzy.

Szablon notatki po rozmowie

```
Potwierdzony osobisty wkład:

Python - wykazane czytanie kodu, proponowana poprawka i dowody projektowe/testowe:

IaC - reakcja na częściowo nieudane wdrożenie:

Platforma danych AWS - przepływ, dostęp i sieć:

Operacje na danych - świeżość, replay i potwierdzenie naprawy:

Komunikacja po angielsku - obserwacje z całej rozmowy:

Główny obszar wymagający wsparcia:

Obszary niesprawdzone:

Odpowiednie doświadczenie - wyjaśnione obowiązki i ustalenia z rekruterem:

Rekomendacja i dowody względem wymagań tego zespołu:
```

5. Pytania rezerwowe - zastępują inne, nie wydłużają rozmowy

Używaj ich, gdy kandydat poruszy temat albo nie możesz przeprowadzić zadania. Zachowaj 60 minut. Nie zastępuj wszystkich pytań o projektowanie i utrzymanie platformy szczegółami z CV.

R8 - ujawnione hasło do bazy w GitHub, 2-4 minuty

Użyj, gdy kandydat pomija temat albo zostaje czas. Bez kodowania.

Zadaj po angielsku:

> You discover that a database password was committed to a GitHub repository used by an AWS data pipeline. A teammate removes it in a new commit and says the problem is solved. Do you agree? What would you do next, and how would you check that the pipeline still works?

Po polsku: hasło do bazy usunięto nowym commitem. Czy to wystarczy i jak sprawdzić działanie platformy?

Minimum na dwie minuty: usunięcie nie wystarcza. Unieważnienie/rotacja ujawnionego poświadczenia z właścicielem systemu, aktualizacja odbiorców i test dozwolonej operacji. Czyszczenie historii nie zastępuje unieważnienia. GitHub: usuwanie danych wrażliwych.

Jeśli masz do czterech minut, wybierz jedno dopytanie:

- “What if the repository is private: does that change the need to act?”
- “The value in Secrets Manager was updated, but the job still fails. What would you check?”
- “What would you change to reduce the chance of this happening again?”

Przejście: “Let's switch to a short scenario about handling secrets.”

Zachowaj minuty 55-60 na pytania kandydata. Zapisz lukę w pominiętym temacie: R8 nie potwierdza tamtej umiejętności. R8 jest opcjonalne.

Zobacz osobny przewodnik R8 i R8 w przykładowych odpowiedziach.

Pozostałe tematy rezerwowe

- **CloudFormation do Terraform:** „Jak przejąć istniejący zasób do zarządzania przez Terraform bez przypadkowego powielenia lub usunięcia?”
- **Testy infrastruktury:** „Czy udany test z uprawnieniami administratora dowodzi, że konto odbiorcy ma właściwe uprawnienia? Jak sprawdzisz operację zabronioną?”
- **Koszt logowania S3:** „W CV porównujesz architektury logowania. Jakie wymagania, wolumeny i koszty porównałeś? Co wdrożono, a co pozostało rekomendacją?”
- **Uzgadnianie danych w SQL:** „Dwa systemy po migracji pokazują różne liczby rekordów. Jak porównasz zbiory, duplikaty i brakujące identyfikatory?”
- **Kontenery:** „Jaki praktyczny problem rozwiązałeś Dockerem i jak diagnozowałeś niedziałający kontener?” Rozwijaj tylko, jeśli temat pasuje do projektu.

Nie przeznaczaj tej godziny na sortowania, metaklasy ani definicje, które nie pomagają ocenić pracy na platformie danych AWS.

Migracja S3 - opcjonalne pogłębienie

> Podczas migracji współistnieją dane historyczne i nowe zapisy. Jak sprawdziłeś, że wszystkie wymagane dane dotarły na miejsce i są czytelne dla odbiorców? Co musiało być spełnione przed przełączeniem?

Wybierz dopytanie:

- Co z obiektami sprzed włączenia replikacji, nadpisaniami i wersjami?
- Dlaczego zgodna liczba obiektów nie dowodzi zgodności zbiorów?
- Jak rozliczysz nieudane kopie i bezpiecznie je ponowisz?

Szukaj oddzielnego traktowania historii i bieżących danych, ustalonego zakresu, raportów braków i testów odbiorcy. ETag nie zawsze jest MD5 danych obiektu; poproś o uzasadnienie metody walidacji. AWS: ETag i sumy kontrolne.

Zwykła replikacja bieżąca nie uzupełnia automatycznie wcześniej istniejących obiektów. Znaczenie mają wersjonowanie i szyfrowanie. AWS: zakres replikacji, AWS: diagnoza replikacji.

Dopytaj przy propozycji AdministratorAccess bez planu diagnozy albo uznawaniu migracji za zakończoną tylko dlatego, że wywołanie API się udało. Najpierw wyjaśnij, czy to skrót myślowy, czy rzeczywiście proponowane działanie.

Rotacja kluczy IAM - opcjonalne pogłębienie

> Dlaczego ten odbiorca potrzebował długoterminowych kluczy IAM zamiast roli i poświadczeń tymczasowych? Przejdź przez proces rotacji. Co się stanie, jeśli Lambda zatrzyma się po utworzeniu klucza, ale przed zapisem w Secrets Manager?

Wybierz dopytanie:

- Jak następne wywołanie rozpozna rozpoczętą rotację i nie utworzy kolejnego klucza?
- Co potwierdzała walidacja STS z CV, a czego nie potwierdzała?
- Kiedy można dezaktywować stary klucz i co, jeśli odbiorca buforuje sekret?

Szukaj uzasadnienia wyjątku od poświadczeń tymczasowych, stanu procesu, identyfikatora rotacji, kontroli współbieżności i nieujawniania sekretów w logach. Sekret klucza IAM można pobrać tylko przy tworzeniu; awaria przed zapisem wymaga świadomej ścieżki odzyskania lub usunięcia osieroconego klucza, a nie obietnicy ponownego odczytania sekretu.

Sekwencja obejmuje nowy klucz, aktualizację i walidację odbiorców, dezaktywację starego klucza i jego późniejsze usunięcie. Limit dwóch kluczy obejmuje także nieaktywne. Udane `GetCallerIdentity` potwierdza tożsamość, a nie dostęp do danych S3 lub bazy. AWS: GetCallerIdentity, AWS: klucze dostępu, AWS: rotacja kluczy.

W krótkim dopytaniu szukaj świadomości częściowych awarii, ponowień i wpływu na odbiorców, a nie kompletnego protokołu rotacji.